

# Default encryption at rest

*This content was last updated in May 2024 and represents the status quo as of the time that it was written. Google's security policies and systems may change going forward, as we continually improve protection for our customers.*

At Google, our comprehensive security strategy includes encryption at rest, which helps to protect customer data from attackers. We encrypt all Google customer content at rest, without any action required by you, using one or more encryption mechanisms. This document describes our approach to default encryption at rest for Google infrastructure and Google Cloud, and how we use it to keep customer content more secure.

This document is for security architects and security teams who are currently using or considering Google. This document assumes a basic understanding of [encryption](https://wikipedia.org/wiki/Encryption) (<https://wikipedia.org/wiki/Encryption>) and [cryptographic primitives](https://wikipedia.org/wiki/Cryptographic_primitive) ([https://wikipedia.org/wiki/Cryptographic\\_primitive](https://wikipedia.org/wiki/Cryptographic_primitive)). For more information on cryptography, see [Introduction to Modern Cryptography](https://www.cs.umd.edu/%7Ejkatz/imc.html) (<https://www.cs.umd.edu/%7Ejkatz/imc.html>).

Encryption at rest is encryption that is used to help protect data that is stored on a disk (including solid-state drives) or backup media. All data that is stored by Google is encrypted at the storage layer using the Advanced Encryption Standard (AES) algorithm, AES-256. We use a common cryptographic library, Tink, which includes our FIPS 140-2 validated module (named [BoringCrypto](https://cloud.google.com/security/compliance/fips-140-2-validated) (<https://cloud.google.com/security/compliance/fips-140-2-validated>)) to implement encryption consistently across Google Cloud.

We own and manage the keys used in default encryption at rest. If you use Google Cloud, Cloud Key Management Service lets you create your own encryption keys that you can use to add envelope encryption to your data. Using Cloud KMS, you can create, rotate, track, and delete keys. For more information, see [Cloud Key Management Service deep dive](/docs/security/key-management-deep-dive) (</docs/security/key-management-deep-dive>).

## Keys in Google Cloud

The following table describes the different properties of keys in Google Cloud.

| Type of key | Cloud KMS Autokey | Cloud KMS customer- | Google-owned and Google-managed encryption key (Google default |
|-------------|-------------------|---------------------|----------------------------------------------------------------|
|-------------|-------------------|---------------------|----------------------------------------------------------------|

|                                                                                                       |                                                                                                   | managed<br>(manual)                 | encryption)                                                                                     |
|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|-------------------------------------|-------------------------------------------------------------------------------------------------|
| Can view key metadata                                                                                 | Yes                                                                                               | Yes                                 | No                                                                                              |
| Ownership of keys <sup>1</sup><br>(#footnote1)                                                        | Customer                                                                                          | Customer                            | Google                                                                                          |
| Can manage <sup>2</sup><br>(#footnote2) and<br>control <sup>3</sup> (#footnote3)<br>keys              | Key creation and<br>assignment is<br>automated. Customer<br>manual control is fully<br>supported. | Customer,<br>manual control<br>only | Google                                                                                          |
| Supports regulatory<br>requirements for<br>customer-managed keys                                      | Yes                                                                                               | Yes                                 | No                                                                                              |
| Key sharing                                                                                           | Unique to a customer                                                                              | Unique to a<br>customer             | Data from multiple customers is typically<br>protected by shared key encryption keys<br>(KEKs). |
| Control of key rotation                                                                               | Yes                                                                                               | Yes                                 | <u>No</u><br>(/docs/security/encryption/default-<br>encryption#kek_management)                  |
| <u>CMEK organization<br/>policies</u><br>(/kms/docs/cmek-org-<br>policy)                              | Yes                                                                                               | Yes                                 | No                                                                                              |
| <u>Log administrative and<br/>data access to encryption<br/>keys</u><br>(/kms/docs/audit-<br>logging) | Yes                                                                                               | Yes                                 | No                                                                                              |
| Logical data separation<br>through encryption                                                         | Yes                                                                                               | Yes                                 | <u>No</u><br>(/docs/security/encryption/default-<br>encryption#hardware)                        |
| Pricing                                                                                               | <u>Varies</u> (/kms/pricing)                                                                      | <u>Varies</u><br>(/kms/pricing)     | Free                                                                                            |

<sup>1</sup> The owner of the key indicates who holds the rights to the key. Keys that you own have tightly restricted access or no access by Google.

<sup>2</sup> Management of keys includes the following tasks:

- Create keys.
- Choose the protection level of the keys.
- Assign authority for management of the keys.
- Control access to keys.
- Control usage of keys.
- Set and modify the rotation period of keys, or trigger a rotation of keys.
- Change key status.
- Destroy key versions.

<sup>3</sup> Control of keys means setting controls on the kind of keys and how the keys are used, detecting variance, and planning corrective action if needed. You can control your keys, but delegate management of the keys to a third party.

## How encryption at rest helps to secure data

Encryption at rest is one piece of a broader security strategy. Encryption has the following benefits:

- Helps to ensure that if data falls into an attacker's hands, the attacker cannot read the data without also having access to the encryption keys. Even if attackers obtain the storage devices that contain customer data, they won't be able to understand or decrypt it.
- Helps to reduce the surface of attack by cutting out the lower layers of the hardware and software stack.
- Acts as a chokepoint because centrally managed encryption keys create a single place where access to data is enforced and can be audited.
- Helps to reduce the attack surface because instead of having to protect all data, businesses can focus their protection strategies on the encryption keys.
- Provides an important privacy mechanism for our customers. When data is encrypted at rest, it limits the access that systems and engineers have to the data

# What is customer data?

As defined in the [Google Cloud Terms of Service](https://cloud.google.com/terms/) (<https://cloud.google.com/terms/>), *customer data* is data that customers or end users provide to Google through the services under their account.

Customer content is data that you generate yourself or provide to us, like data stored in Cloud Storage buckets, Persistent Disk volumes, and disk snapshots used by Compute Engine. This document focuses on default encryption at rest for this type of customer data.

Customer metadata is data about your customer content and includes auto-generated project numbers, timestamps, IP addresses, the byte size of an object in Cloud Storage, or the machine type in Compute Engine. Customer metadata is protected to a degree that is reasonable for ongoing performance and operations. This document doesn't focus on the protections for metadata.

Together, customer content and customer metadata make up customer data.

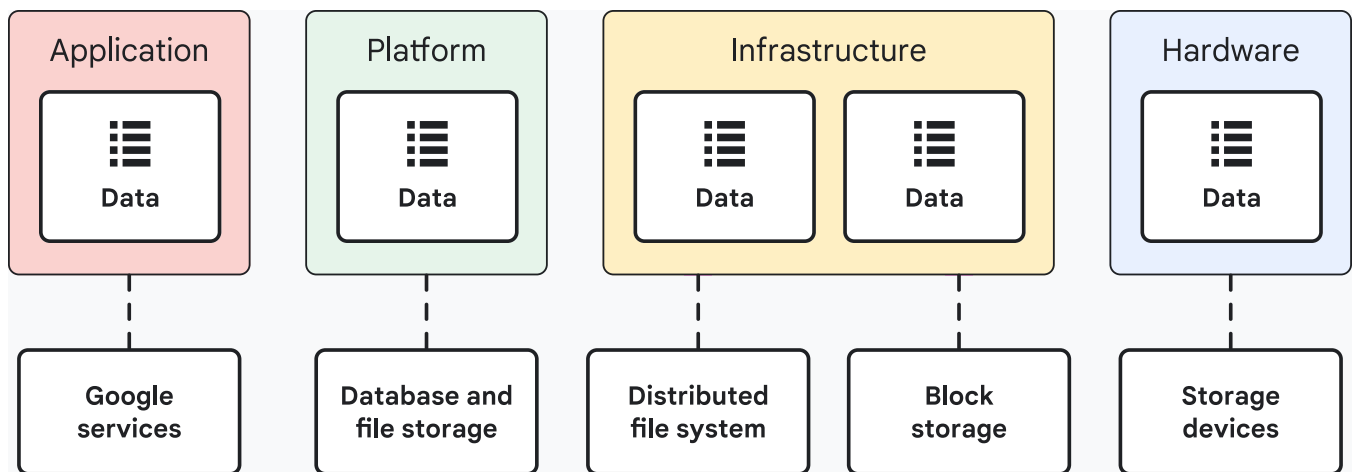
## Default encryption of data at rest

Google encrypts all customer content stored at rest, without any action from you, using one or more encryption mechanisms. The following sections describe the mechanisms that we use to encrypt customer content.

### Layers of encryption

Google uses several layers of encryption to help protect data. Using multiple layers of encryption adds redundant data protection and allows us to select the optimal approach based on application requirements.

The following diagram shows the several layers of encryption that are generally used to protect user data in Google production data centers. Either distributed file system encryption or database and file storage encryption is in place for all user data, and storage device encryption is in place for all data in Google production data centers.



## Encryption at the infrastructure layer

All of Google's storage systems use a similar encryption architecture, though implementation details differ from system to system. Data is broken into logical chunks for storage; each chunk can be up to several gigabytes in size. Each chunk is encrypted at the storage level with an individual data encryption key (DEK): two chunks won't have the same DEK, even if they are owned by the same customer or stored on the same machine.

If a chunk of data is updated, it is encrypted with a new key, rather than by reusing the existing key. This partitioning of data, each using a different key, limits the risk of a potential data encryption key compromise to only that data chunk.

Google encrypts data before it is written to a database storage system or hardware disk. Encryption is inherent in all of our storage systems, rather than added afterward.

Each logical data chunk has a unique identifier. Access control lists (ACLs) help to ensure that each chunk can be decrypted only by Google services that operate with authorized roles, which are granted access only at that point in time. This access limitation helps to prevent access to the data without authorization, strengthening data security and privacy.

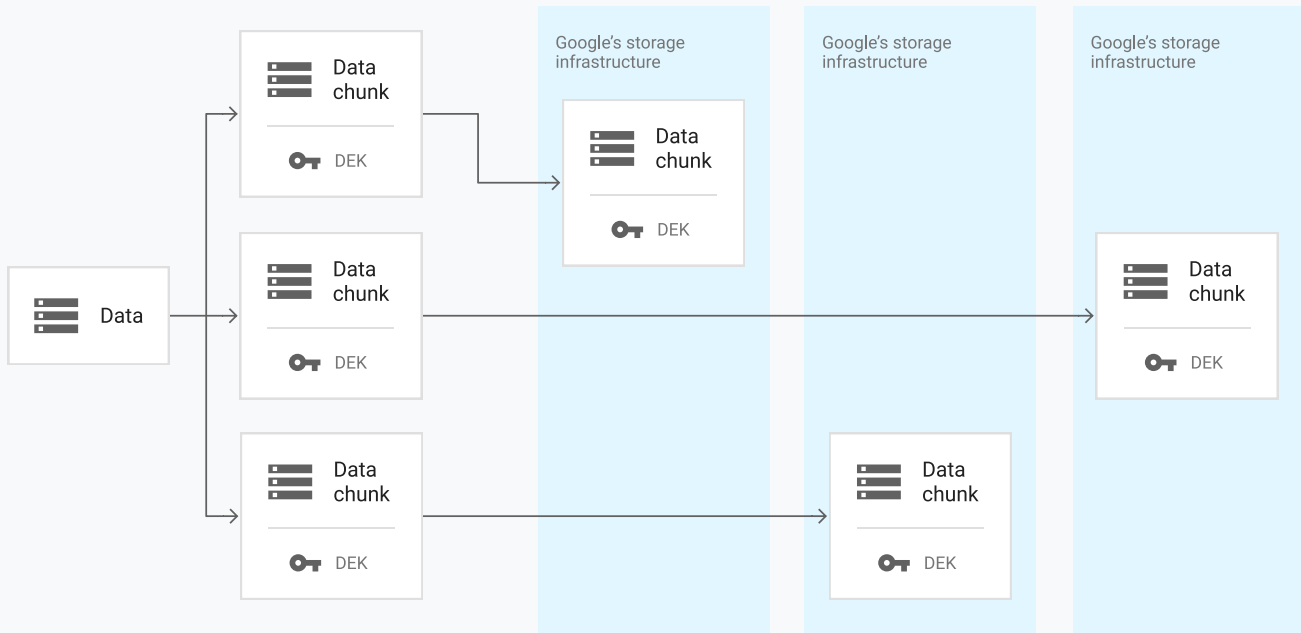
Each chunk is distributed across our storage systems and is replicated in encrypted form for backup and disaster recovery. An attacker who wants to access customer data would need to know and be able to access two things: all of the storage chunks that correspond to the data that they want and all of the encryption keys that correspond to the chunks.

The following diagram shows how data is uploaded to our infrastructure and then broken into encrypted chunks for storage.

Data is uploaded to Google

Data is chunked and each chunk is encrypted with its own key

Chunks are distributed across Google's storage infrastructure



We use the AES algorithm to encrypt data at rest. All data at the storage level is encrypted by DEKs, which use AES-256 by default, with the exception of a small number of Persistent Disks (<https://cloud.google.com/persistent-disk>) that were created before 2015 that use AES-128. AES is widely used because both AES-256 and AES-128 are recommended by the National Institute of Standards and Technology (NIST). (<https://csrc.nist.gov/publications/detail/sp/800-131a/rev-2/final>) for long-term storage use, and AES is often included as part of customer compliance requirements.

A logical data chunk might contain the data of multiple customers. If you want to achieve logical data separation through encryption, you must enable Cloud Key Management Service (</docs/security/key-management-deep-dive#cloud-kms-design-pillars>).

## Encryption at the storage device layer

In addition to storage system level encryption (#hardware), data is also encrypted at the storage device level with AES-256 for hard disk drives (HDD) and solid-state drives (SSD), using a separate device-level key (which is different from the key used to encrypt the data at the storage level). A small number of legacy HDDs use AES-128. SSDs used by Google implement AES-256 for user data exclusively.

## Encryption of backups

Our backup system ensures that data remains encrypted throughout the backup process. This approach avoids unnecessarily exposing plaintext data.

In addition, the backup system further encrypts most backup files independently with their own DEK. The DEK is derived from a key that is stored in Keystore and a randomly generated per-file seed at backup time. Another DEK is used for all metadata in backups, which is also stored in Keystore.

## FIPS compliance for data at rest

Google uses a FIPS 140-2 validated

(<https://cloud.google.com/security/compliance/fips-140-2-validated>) encryption module (certificate 4407) (<https://csrc.nist.gov/projects/cryptographic-module-validation-program/certificate/4407>) in our production environment.

## Key management

Because of the high volume of keys at Google, and the need for low latency and high availability, DEKs are stored near the data that they encrypt. DEKs are encrypted with (wrapped by) a key encryption key (KEK), using a technique known as envelope encryption (</kms/docs/envelope-encryption>). These KEKs are not specific to customers; instead, one or more KEKs exist for each service.

These KEKs are stored centrally in Keystore, a repository built specifically for storing keys. Having a smaller number of KEKs than DEKs and using a central Keystore makes storing and encrypting data at our scale manageable, and lets us track and control data access from a central point.

**Note:** Keystore was formerly known as *Google's key management service*. It is different from Cloud KMS, which manages the encryption keys for Google Cloud customers and helps customers to create their tenant keys.

In Google Cloud, each customer can have shared and non-shared resources. An example of a shared resource is a shared base image in Compute Engine. For shared resources, multiple

customers refer to a single copy, which is encrypted by a single DEK. Non-shared resources are split into data chunks and encrypted with keys that are separate from the keys used for other customers. These keys are even separate from those that protect other pieces of the same data owned by that same customer. There are exceptions (for example, Datastore, App Engine, or Pub/Sub) where more than one customer's data might be encrypted with the same DEK.

## Generating DEKs

The storage system generates DEKs using Google's common cryptographic library. In general, DEKs are then sent to Keystore to wrap with that storage system's KEK, and the wrapped DEKs are passed back to the storage system to be kept with the data chunks. When a storage system needs to retrieve encrypted data, it retrieves the wrapped DEK and passes it to Keystore. Keystore then verifies that this service is authorized to use the KEK and, if so, unwraps and returns the plaintext DEK to the service. The service then uses the DEK to decrypt the data chunk into plaintext and verify its integrity.

All Google Cloud storage systems adhere to this key management model, but most systems also implement additional levels of storage-side KEKs to create a hierarchy of keys. This allows the systems to provide low latency while using the highest-level KEK (stored in Keystore) as their root of trust.

## Generating KEKs

Most KEKs for encrypting data chunks are generated within Keystore, and the rest are generated inside the storage services. For consistency, all KEKs are generated using Google's common cryptographic library, using a random number generator (RNG) built by Google. This RNG is based on NIST 800-90Ar1 CTR-DRBG and generates an AES-256 KEK. (In the past, this was AES-128, and some of these keys remain active for decrypting data.)

For Intel and AMD processors, the RNG is seeded from RDRAND instruction

(<https://software.intel.com/en-us/articles/intel-digital-random-number-generator-drng-software-implementation-guide>)

and the Linux kernel's RNG. In turn, the Linux kernel's RNG is seeded from multiple independent entropy sources, including RDRAND and entropic events from the data center environment (for example, fine-grained measurements of disk seeks and inter-packet arrival times). For Arm processors, the RNG is seeded from the Linux kernel's RNG.

DEKs are wrapped with KEKs using AES-256 or AES-128, depending on the Google Cloud service. We are currently working on upgrading all KEKs for Google Cloud services to AES-256.

## KEK management

Keystore was built solely for the purpose of managing KEKs. By design, KEKs used by storage systems aren't exportable from Keystore; all encryption and decryption with these keys must be done within Keystore. This helps to prevent leaks and misuse, and it enables Keystore to create an audit trail when keys are used.

Keystore can automatically rotate KEKs at regular time intervals, using Google's common cryptographic library to generate new keys. Though we often refer to just a single key, we really mean that data is protected using a key set: one key is active for encryption, and a set of historical keys is active for decryption. The number of historical keys is determined by the key rotation schedule. KEKs are backed up for disaster recovery purposes, and they are indefinitely recoverable.

The use of KEKs is managed by ACLs in Keystore for each key, with a per-key policy. Only authorized Google services and users are allowed to access a key. The use of each key is tracked at the level of the individual operation that requires that key—so every time that a user uses a key, the user is authenticated and logged. All data access by users is auditable as part of Google's overall security and privacy policies.

## Process for accessing encrypted chunks of data

When a Google service accesses an encrypted chunk of data, the following occurs:

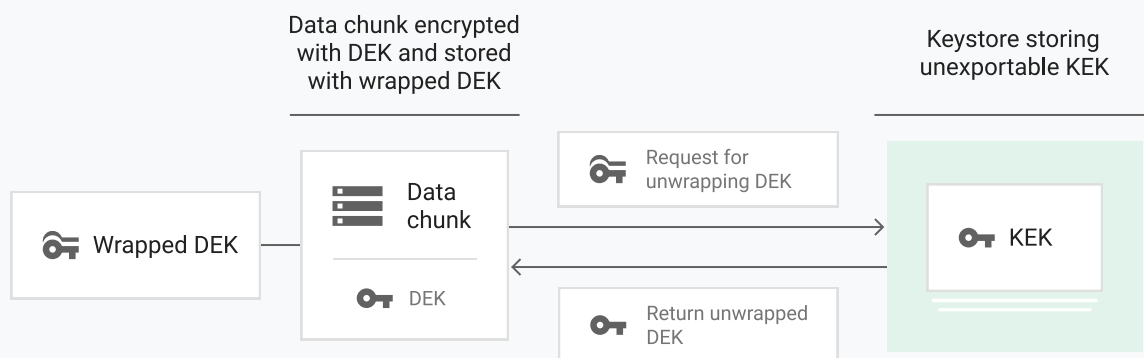
1. The service makes a call to the storage system for the data that it needs.
2. The storage system identifies the chunks in which that data is stored (the chunk IDs) and where they are stored.
3. For each chunk, the storage system pulls the wrapped DEK that is stored with that chunk (in some cases, this is done by the service) and sends it to Keystore for unwrapping.
4. The storage system verifies that the identified job is allowed to access that data chunk based on a job identifier and using the chunk ID. Keystore verifies that the storage system is authorized to use the KEK that is associated with the service and to unwrap that specific DEK.

5. Keystore does one of the following:

- Passes the unwrapped DEK back to the storage system, which decrypts the data chunk and passes it to the service.
- In some rare cases, passes the unwrapped DEK to the service. The storage system passes the encrypted data chunk to the service, which decrypts the data chunk and uses it.

This process is different in dedicated storage devices, where the device manages and protects the device-level DEK.

The following diagram shows this process. To decrypt a data chunk, the storage service calls Keystore to retrieve the unwrapped DEK for that data chunk.



## Encryption key hierarchy and root of trust

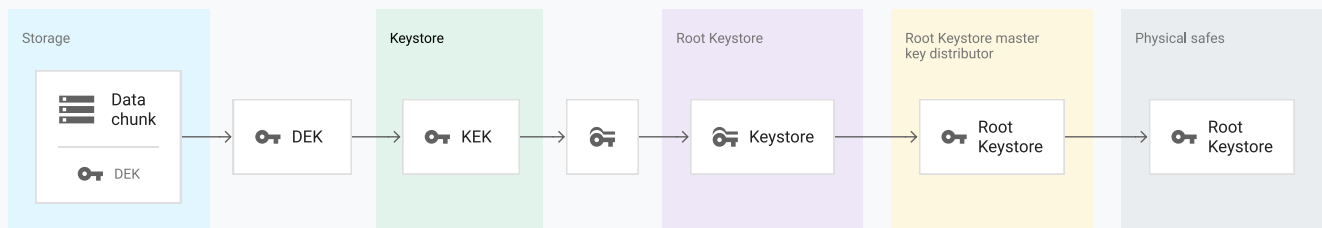
Keystore is protected by a root key called the *keystore master key*, which wraps all of the KEKs in Keystore. This keystore master key is AES-256 and is itself stored in another key management service, called Root Keystore. (In the past, the keystore master key was AES-128, and some of these keys remain active for decrypting data.) For additional security, Root Keystore isn't run on general production machines, but instead is run only on dedicated machines in each Google data center.

Root Keystore in turn has its own root key, called the *root keystore master key*, which is also AES-256 and is stored in a peer-to-peer infrastructure, which is called the *root keystore master key distributor*, and which replicates these keys globally. (In the past, the root keystore master key was AES-128, and some of these keys remain active for decrypting data.) The root keystore master key distributor only holds the keys in RAM on the same dedicated machines as Root Keystore, and it uses logging to verify proper use.

When a new instance of the root keystore master key distributor is started, it is configured with a list of host names of already running distributor instances. Distributor instances can then obtain the root keystore master key from other running instances. Other than the disaster-recovery mechanisms described in [Global availability and replication](#) (#global), the root keystore master key exists only in RAM on a limited number of specially secured machines.

To address the scenario where all instances of the root keystore master key distributor in a region restart simultaneously, the root keystore master key is also backed up on secure hardware devices that are stored in physical safes in highly secured areas in multiple geographically distributed locations. This backup would be needed only if all distributor instances in a region were to go down at once. Only a few Google employees can access these safes.

The following diagram shows the encryption key hierarchy. The encryption key hierarchy protects a chunk of data with a DEK, wrapped with a KEK in Keystore, which is in turn protected by Root Keystore and the root keystore master key distributor.



## Summary of key management

The following list summarizes key management at Google:

- Data is chunked and encrypted with DEKs.
- DEKs are encrypted with KEKs.
- KEKs are stored in Keystore.
- Keystore is run on multiple machines in data centers globally.
- Keystore keys are wrapped with the Keystore master key, which is stored in Root Keystore.
- Root Keystore is much smaller than Keystore and runs only on dedicated machines in each data center.

- Root Keystore keys are wrapped with the root keystore master key, which is stored in the root keystore master key distributor.
- The Root Keystore master key distributor is a peer-to-peer infrastructure that runs concurrently in RAM on dedicated machines globally. Each machine gets its key material from other running instances in the region.
- In case all instances of the distributor in a region were to go down, a master key is stored in different secure hardware in physical safes in limited Google locations.

## Global availability and replication

At every level, high availability, low latency, and global access to keys are critical. These characteristics are needed for key management services to be used across Google.

For this reason, Keystore is highly scalable, and it is replicated thousands of times in our data centers globally. It is run on regular machines in our production fleet, and instances of Keystore run globally to support Google operations. As a result, the latency of any single key operation is very low.

Root Keystore is run on several machines dedicated to security operations in each data center. The Root Keystore master key distributor is run on these same machines, one-to-one with Root Keystore. The Root Keystore master key distributor provides a distribution mechanism using a [gossiping protocol](https://dl.acm.org/citation.cfm?doid=41840.41841) (<https://dl.acm.org/citation.cfm?doid=41840.41841>). At a fixed time interval, each instance of the distributor picks a random other instance to compare its keys with and reconciles any differences in key versions. With this model, there is no central node that all of our infrastructure depends on. This distribution method lets us maintain and protect key material with high availability.

## Google's common cryptographic library

Google's common cryptographic library is [Tink](https://developers.google.com/tink) (<https://developers.google.com/tink>), which incorporates our FIPS 140-2 validated module, [BoringCrypto](https://cloud.google.com/security/compliance/fips-140-2-validated) (<https://cloud.google.com/security/compliance/fips-140-2-validated>). Tink is available to all Google developers. Consistent use of a common library means that only a small team of cryptographers needs to implement this tightly controlled and reviewed code, making it unnecessary for every team at Google to independently develop their own cryptography. A

special Google security team is responsible for maintaining this common cryptographic library for all products.

The Tink encryption library supports a wide variety of encryption key types and modes, and these are reviewed regularly to ensure that they are current with the latest attacks.

**Note:** Google also uses another library called Keymaster. Keymaster shares newer cryptography code in common with Tink, but Keymaster uses a different key-versioning implementation and supports a wider variety of older algorithms.

Currently, we use the following encryption algorithms for encryption at rest for DEKs and KEKs. These are subject to change as we continue to improve our capabilities and security.

| Cryptographic primitive                                                             | Preferred protocols | Other supported protocols                                                                                                     |
|-------------------------------------------------------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Symmetric encryption                                                                | AES-GCM (256 bits)  | <ul style="list-style-type: none"><li>• AES-CBC and AES-CTR (128 and 256 bits)</li><li>• AES-EAX (128 and 256 bits)</li></ul> |
| Symmetric signatures (where used with AES-CBC and AES-CTR above for authentication) | HMAC-SHA256         | <ul style="list-style-type: none"><li>• HMAC-SHA512</li><li>• HMAC-SHA1</li></ul>                                             |

Other cryptographic protocols exist in the library and were historically supported, but this table covers the primary uses at Google.

## Research and innovation in cryptography

To keep pace with the evolution of encryption, we have a team of world-class security engineers tasked with following, developing, and improving encryption technology. Our engineers take part in standardization processes and in maintaining widely used encryption software. We regularly publish our research (<https://www.google.com/about/appsecurity/research/>) in the field of encryption so that everyone—including the general public—can benefit from our knowledge.

For example, in post-quantum cryptography research, we are working in the following areas:

- **Standardization:** We co-designed the stateless hash-based digital signature scheme that is standardized as [FIPS 205](https://csrc.nist.gov/pubs/fips/205/ipd) (<https://csrc.nist.gov/pubs/fips/205/ipd>). We are editors of the International Organization for Standardization (ISO) standard on [post-quantum cryptography hash-based signatures](https://www.iso.org/standard/80492.html) (<https://www.iso.org/standard/80492.html>) and contributed to [guidance on state management](https://www.ietf.org/archive/id/draft-wiggers-hbs-state-00.txt) (<https://www.ietf.org/archive/id/draft-wiggers-hbs-state-00.txt>) for hash-based signatures at IETF.
- **Enablement:** We [rolled out post-quantum cryptography](https://cloud.google.com/blog/products/identity-security/why-google-now-uses-post-quantum-cryptography-for-internal-comms/) (<https://cloud.google.com/blog/products/identity-security/why-google-now-uses-post-quantum-cryptography-for-internal-comms/>) to our internal protocol for transport layer security. We enabled support for post-quantum cryptography in [Chrome](https://blog.chromium.org/2023/08/protecting-chrome-traffic-with-hybrid.html) (<https://blog.chromium.org/2023/08/protecting-chrome-traffic-with-hybrid.html>). We added several post-quantum cryptography algorithms in our [Tink cryptographic library](https://github.com/google/tink/tree/master/cc/experimental/pqcrypto) (<https://github.com/google/tink/tree/master/cc/experimental/pqcrypto>). This code is experimental and is designed to help educate the community about each approach.
- **Publications:** We published [Transitioning organizations to post-quantum cryptography](https://www.nature.com/articles/s41586-022-04623-2) (<https://www.nature.com/articles/s41586-022-04623-2>) in *Nature*. This paper provides an overview of post-quantum cryptography migration challenges. We also published a research paper on getting post-quantum cryptography in [our security keys](https://security.googleblog.com/2023/08/toward-quantum-resilient-security-keys.html) (<https://security.googleblog.com/2023/08/toward-quantum-resilient-security-keys.html>).

Note that symmetric encryption (using AES-128 or later) remains resistant to quantum attacks.

## What's next

- For information about Google's post-quantum cryptography resources, see [Post-quantum cryptography](https://cloud.google.com/security/resources/post-quantum-cryptography) (<https://cloud.google.com/security/resources/post-quantum-cryptography>).
- For information about using your own encryption keys in Google Cloud, see [Customer-managed encryption keys \(CMEK\)](https://cloud.google.com/kms/docs/cmek) ([/kms/docs/cmek](https://cloud.google.com/kms/docs/cmek)).
- For general information about Google Cloud security, see the [Security section of the Google Cloud website](https://cloud.google.com/security/) (<https://cloud.google.com/security/>).
- For information about Google Cloud compliance and compliance certifications, see the [Compliance section of the Google Cloud website](https://cloud.google.com/compliance/) (<https://cloud.google.com/compliance/>).

(<https://cloud.google.com/security/compliance>), which includes Google's public SOC3 audit report (<https://www.google.com/work/soc3.html>).

- For information about Google Workspace encryption and key management, see How Google Workspace uses encryption to protect your data

(<https://storage.googleapis.com/gfw-touched-accounts-pdfs/google-encryption-whitepaper-gsuite.pdf>)

, which covers much of the same content included here, but focuses solely on Google Workspace.

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the Apache 2.0 License (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the Google Developers Site Policies (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-08-26 UTC.